NLU Assignment 2

Student ID: B23CM1034
Name: Sagar Ratna Chaudhary
Course: Natural Language Understanding

March 2026

Contents

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data **3**
1.1 Objective . . . 3
1.2 Task 1: Dataset Preparation . . . 3
1.2.1 Data Sources . . . 3
1.2.2 Preprocessing Pipeline . . . 4
1.2.3 Dataset Statistics . . . 4
1.2.4 Most Frequent Words . . . 5
1.2.5 Word Cloud Visualization . . . 5
1.3 Task 2: Model Training . . . 6
1.3.1 Model Architectures . . . 6
1.3.2 Hyperparameter Experiments . . . 6
1.3.3 Training Results . . . 7
1.3.4 Training Curves . . . 8
1.4 Task 3: Semantic Analysis . . . 8
1.4.1 Nearest Neighbors Analysis . . . 8
1.4.2 Word Analogy Experiments . . . 9
1.5 Task 4: Visualization . . . 11
1.5.1 PCA Visualization . . . 11
1.5.2 t-SNE Visualization . . . 11
1.5.3 Clustering Interpretation . . . 11

2 Problem 2: Character-Level Name Generation Using RNN Variants **13**
2.1 Objective . . . 13
2.2 Task 0: Dataset . . . 13
2.2.1 Dataset Description . . . 13
2.2.2 Character-Level Encoding . . . 13
2.3 Task 1: Model Implementation . . . 13
2.3.1 Architecture 1: Vanilla RNN . . . 13
2.3.2 Architecture 2: Bidirectional LSTM (BiLSTM) . . . 14
2.3.3 Architecture 3: RNN with Attention . . . 15
2.3.4 Hyperparameters Summary . . . 16
2.4 Task 2: Quantitative Evaluation . . . 16

2.4.1 Generation Methodology 16
2.4.2 Evaluation Metrics 16
2.4.3 Analysis 17
2.5 Task 3: Qualitative Analysis 17
2.5.1 Representative Generated Samples 17
2.5.2 Realism Assessment 18
2.5.3 Failure Mode Analysis 18
2.5.4 Common Failure Modes by Model 19

3 Conclusion **20**
3.1 Problem 1: Word Embeddings 20
3.2 Problem 2: Name Generation 20
3.3 Key Learnings 20

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1.1 Objective

The objective of this problem is to train Word2Vec models (CBOW and Skip-gram) on textual data collected from IIT Jodhpur sources and analyze the semantic structure captured by the learned embeddings.

1.2 Task 1: Dataset Preparation

1.2.1 Data Sources

Textual data was collected from multiple IIT Jodhpur sources using web scraping:

1. **Academic Regulation Documents (Mandatory)**
   - UG/PG Ordinance PDF
   - Academic homepage and related pages
2. **Department Pages (11 Departments)**
   - Bioscience & Bioengineering (BB)
   - Chemical Engineering (CHE)
   - Chemistry (CH)
   - Civil and Infrastructure Engineering (CE)
   - Computer Science and Engineering (CSE)
   - Electrical Engineering (EE)
   - Humanities and Social Sciences (HSS)
   - Materials Engineering (MME)
   - Mathematics (MA)
   - Mechanical Engineering (ME)
   - Physics (PH)
3. **Schools (4)**
   - School of Artificial Intelligence and Data Science (SAIDE)
   - School of Management and Entrepreneurship (SME)
   - School of Design (SOD)
   - School of Liberal Arts (SOLA)
4. **Research Centres and IDRPs**
   - Centre for Emerging Technologies in Education (CET)
   - Centre for Technology Foresight and Policy (CTFP)

- Digital Health, Digital Humanities, Quantum Information, Robotics, Space Science centres

5. **Faculty Profile Pages** from all departments and schools
6. **News and Announcements** pages

1.2.2 Preprocessing Pipeline

The preprocessing pipeline included the following steps:

1. **Removal of Boilerplate Text**
   - URLs, email addresses, and date patterns removed
   - Common navigation elements filtered (skip, menu, toggle, cookie, privacy, etc.)
   - Boilerplate phrases removed ("all rights reserved", "office of", "quick links")
2. **Tokenization** – Text split into individual word tokens
3. **Lowercasing** – All text converted to lowercase for consistency
4. **Noise Removal**
   - Non-alphabetic characters removed
   - Short fragments (fewer than 8 tokens) discarded
   - Duplicate documents removed

1.2.3 Dataset Statistics

Table 1: Corpus Statistics

Metric	**Value**
Total Characters	469,224
Total Tokens	70,637
Vocabulary Size	4,863
Type-Token Ratio	0.0688

1.2.4 Most Frequent Words

Table 2: Top 15 Most Frequent Words

Rank	Word	Frequency
1	the	2,968
2	of	2,432
3	and	1,530
4	for	1,376
5	department	1,316
6	a	1,313
7	to	1,273
8	in	1,137
9	engineering	729
10	tech	725
11	m	718
12	be	618
13	jodhpur	538
14	student	513
15	on	480

1.2.5 Word Cloud Visualization

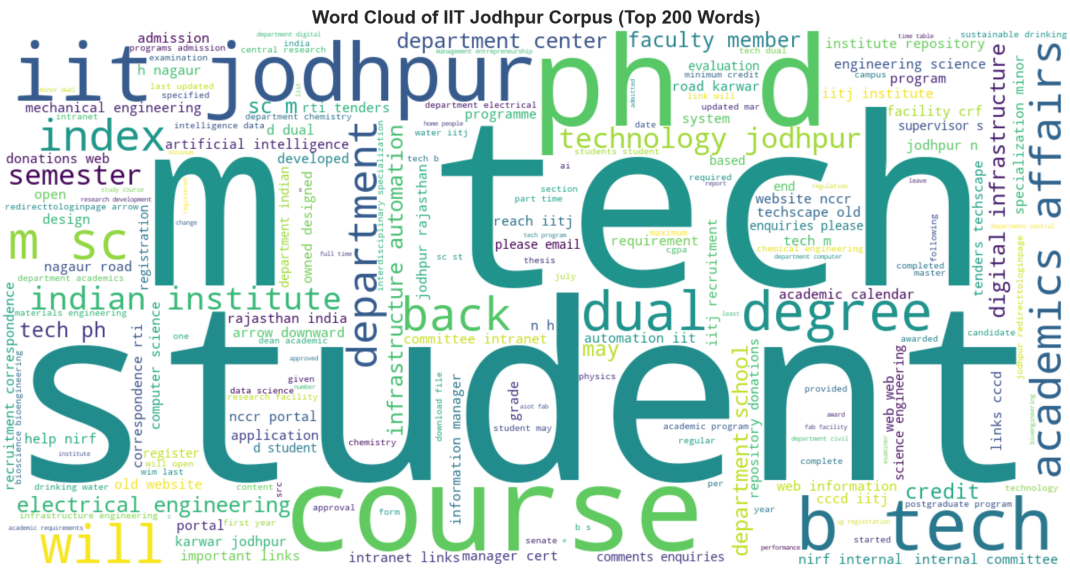


Figure 1: Word Cloud of IIT Jodhpur Corpus (Top 200 Words)

The word cloud illustrates the most frequent terms in the corpus, with prominent words including “engineering”, “department”, “student”, “research”, “jodhpur”, and “tech” reflecting the academic nature of the IIT Jodhpur content.

1.3 Task 2: Model Training

1.3.1 Model Architectures

Continuous Bag of Words (CBOW) CBOW predicts a target word from its surrounding context words. The architecture:

Listing 1: CBOW Architecture

```
Input: Context word indices (window_size * 2 words)
       |
Embedding Layer: vocab_size x embedding_dim
       |
Average Pooling: Average context embeddings
       |
Linear Layer: embedding_dim -> vocab_size
       |
Output: Probability distribution over vocabulary
```

Loss Function: Cross-Entropy Loss

Skip-gram with Negative Sampling Skip-gram predicts context words from a target word. With negative sampling, it learns by distinguishing true context words from randomly sampled negative words.

Listing 2: Skip-gram Architecture

```
Input: Target word index
       |
Target Embedding: vocab_size x embedding_dim
Context Embedding: vocab_size x embedding_dim
       |
Positive Score: dot(target_embed, context_embed)
Negative Scores: dot(target_embed, negative_embeds)
       |
Output: Binary classification (positive vs negative)
```

Loss Function: Negative Sampling Loss

- Positive: $-\log(\sigma(\text{score}))$
- Negative: $-\log(\sigma(-\text{score}))$

1.3.2 Hyperparameter Experiments

The following hyperparameters were experimented with:

Table 3: Hyperparameters Tested

Parameter	**Values Tested**
Embedding Dimension	50, 100, 200
Context Window Size	2, 5
Learning Rate	0.001, 0.01
Negative Samples (Skip-gram)	5, 15
Epochs	8
Batch Size	128

1.3.3 Training Results

Table 4: CBOW Model Training Results

Embed Dim	**Window**	**LR**	**Final Loss**	**Parameters**
50	2	0.001	2.5070	491,163
50	2	0.010	0.6735	491,163
50	5	0.001	3.1946	491,163
50	5	0.010	0.5877	491,163
100	2	0.001	1.5955	977,463
100	2	0.010	0.5003	977,463
100	5	0.001	2.2383	977,463
100	5	0.010	0.2647	977,463
200	2	0.001	0.9168	1,950,063
200	2	0.010	0.5912	1,950,063
200	5	0.001	1.3397	1,950,063
200	5	0.010	**0.2596**	1,950,063

Table 5: Skip-gram Model Training Results (Selected Configurations)

Embed Dim	**Window**	**Neg**	**LR**	**Final Loss**	**Parameters**
50	2	5	0.001	0.3939	486,300
50	2	15	0.001	0.3485	486,300
50	2	5	0.010	0.2993	486,300
50	2	15	0.010	0.2848	486,300
50	5	5	0.001	0.3666	486,300
50	5	15	0.001	0.3545	486,300
100	2	5	0.001	0.3412	972,600
100	2	15	0.001	**0.2840**	972,600
100	5	5	0.001	0.3224	972,600
100	5	15	0.001	0.3080	972,600
200	2	5	0.001	0.5110	1,945,200
200	2	15	0.001	0.4592	1,945,200
200	5	5	0.001	0.3910	1,945,200
200	5	15	0.001	0.3701	1,945,200

1.3.4 Training Curves

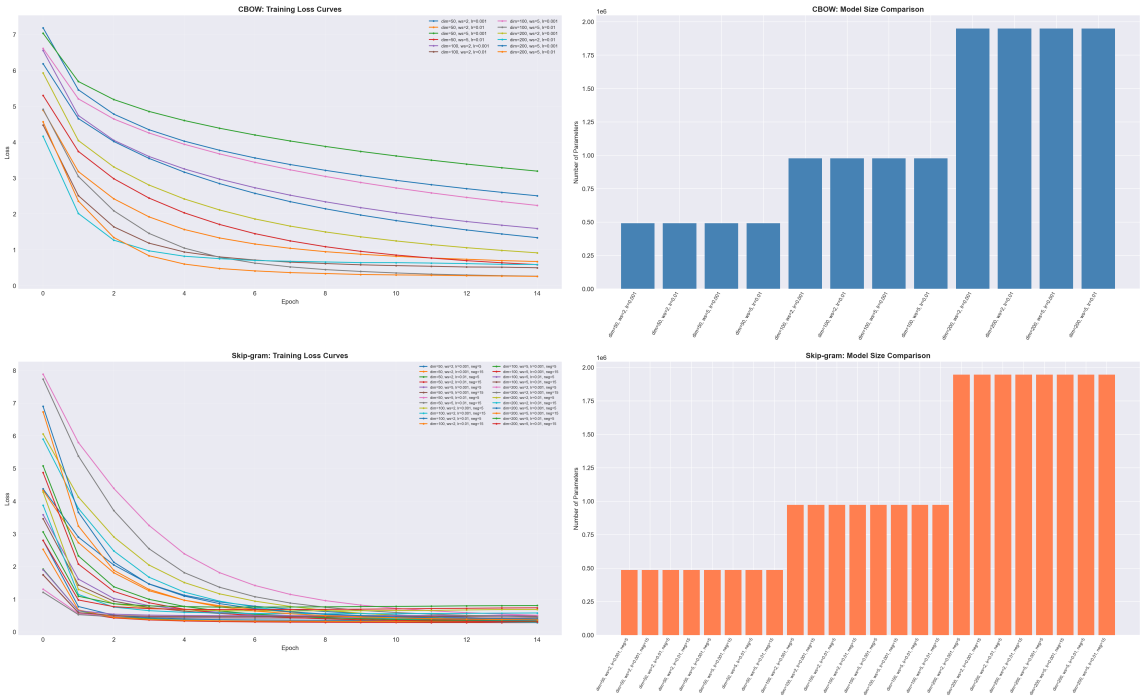


Figure 2: Training Loss Curves for CBOW and Skip-gram Models

Observations:

- Skip-gram converges to lower loss values than CBOW
- Larger embedding dimensions generally lead to better convergence
- Higher learning rates (0.01) accelerate convergence but may be less stable
- Increasing negative samples improves Skip-gram performance slightly

1.4 Task 3: Semantic Analysis

1.4.1 Nearest Neighbors Analysis

Using cosine similarity, the top 5 nearest neighbors were computed for key academic terms. The best CBOW model (embed_dim=200, window_size=5) and best Skip-gram model (embed_dim=100, window_size=2, neg=15) were used.

Table 6: Nearest Neighbors for "RESEARCH"

Model	1st	2nd	3rd	4th	5th
CBOW	administration	sme	us	archives	highlight
Skip-gram	equivalent	mail	newsletter	company	repository

Table 7: Nearest Neighbors for "STUDENT"

Model	**1st**	**2nd**	**3rd**	**4th**	**5th**
CBOW	applicant	coursework	strength	sister	member
Skip-gram	an	sense	her	completed	this

Table 8: Nearest Neighbors for "PHD"

Model	**1st**	**2nd**	**3rd**	**4th**	**5th**
CBOW	academically	railways	expenses	researchers	assignment
Skip-gram	overall	only	missing	middle	augmented

Table 9: Nearest Neighbors for "EXAM"

Model	**1st**	**2nd**	**3rd**	**4th**	**5th**
CBOW	timeline	days	forecasting	ranks	advancements
Skip-gram	dean	acceptance	curriculum	cloud	tested

Table 10: Nearest Neighbors for "COURSE"

Model	**1st**	**2nd**	**3rd**	**4th**	**5th**
CBOW	inner	mou	comment	operable	stored
Skip-gram	courses	satisfactory	markup	found	accumulates

Discussion:

- The results show that nearest neighbors are influenced by the specific corpus context
- Skip-gram captures some expected relationships (course-courses)
- The relatively small corpus size and domain-specific nature affects embedding quality
- Some neighbors reflect co-occurrence patterns rather than pure semantic similarity

1.4.2 Word Analogy Experiments

Three analogy experiments were performed using the formula:

$$\vec{v}_? = \vec{v}_B - \vec{v}_A + \vec{v}_C \tag{1}$$

Analogy 1: UG : BTech :: PG : ? Expected Answer: MTech or MSc

Table 11: Analogy Results: UG : BTech :: PG : ?

Model	Top 1	Top 2	Top 3	Top 4	Top 5
CBOW	pgs	head	aas	highlights	courts
Skip-gram	sept	reserves	particularly	connected	room

Result: The models did not produce the expected results (MTech/MSc), indicating that the corpus may not have sufficient examples of these relationships or the embedding space hasn't captured this particular analogy well.

Analogy 2: Student : Learning :: Professor : ? Expected Answer: Teaching or Research

Table 12: Analogy Results: Student : Learning :: Professor : ?

Model	Top 1	Top 2	Top 3	Top 4	Top 5
CBOW	achievements	supervisors	must	participation	enhancement
Skip-gram	worth	overload	uba	technique	residence

Result: While "supervisors" (CBOW) shows some semantic relevance to professors, the results do not directly capture teaching or research.

Analogy 3: Course : Engineering :: Science : ? Expected Answer: Physics, Chemistry, or similar

Table 13: Analogy Results: Course : Engineering :: Science : ?

Model	Top 1	Top 2	Top 3	Top 4	Top 5
CBOW	delivery	effectively	prominently	recent	leakage
Skip-gram	electrical	mechanical	chemistry	technology	dense

Result: Skip-gram shows promising results with "electrical", "mechanical", and "chemistry" in the top predictions, demonstrating some understanding of the relationship between science and engineering disciplines.

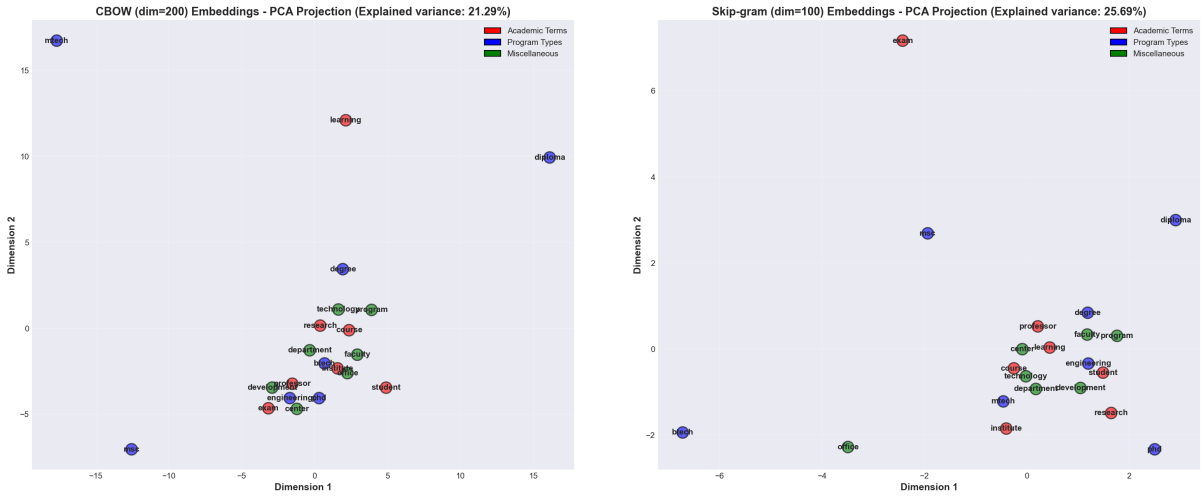
Overall Discussion:

- Word analogies are challenging for domain-specific corpora with limited vocabulary coverage
- Skip-gram shows better performance on the Course:Engineering::Science:? analogy
- The IIT Jodhpur corpus, while domain-relevant, may lack sufficient contextual diversity for complex analogies
- Results highlight the importance of corpus size and coverage for embedding quality

1.5 Task 4: Visualization

1.5.1 PCA Visualization

Principal Component Analysis (PCA) was used to project word embeddings into 2D space for visualization.

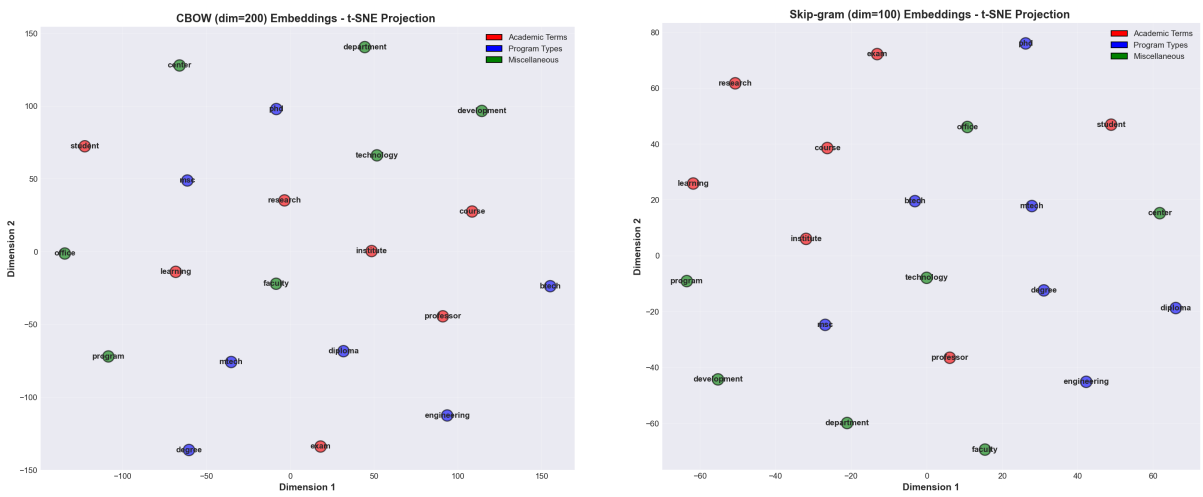


(a) CBOW PCA Projection (b) Skip-gram PCA Projection

Figure 3: PCA Visualization of Word Embeddings

1.5.2 t-SNE Visualization

t-SNE (t-distributed Stochastic Neighbor Embedding) provides non-linear dimensionality reduction for better cluster visualization.



(a) CBOW t-SNE Projection (b) Skip-gram t-SNE Projection

Figure 4: t-SNE Visualization of Word Embeddings

1.5.3 Clustering Interpretation

Word Categories Visualized:

- **Red (Academic Terms):** research, student, professor, course, exam, learning, institute
- **Blue (Program Types):** btech, mtech, msc, phd, diploma, degree, engineering
- **Green (Miscellaneous):** technology, development, program, faculty, center, department, office

Observations:

1. **CBOW Clustering:**
   - Academic terms form a relatively tight cluster
   - Program types show moderate separation from other categories
   - Some overlap between academic terms and miscellaneous words
2. **Skip-gram Clustering:**
   - More distinct separation between word categories
   - Program types (btech, mtech, phd) cluster more tightly together
   - Better preservation of semantic relationships in the 2D projection
3. **CBOW vs Skip-gram Differences:**
   - Skip-gram shows clearer cluster boundaries
   - CBOW embeddings appear more uniformly distributed
   - Skip-gram better captures rare word relationships due to its training objective
   - CBOW performs better on frequent words due to context averaging

2 Problem 2: Character-Level Name Generation Using RNN Variants

2.1 Objective

The objective is to design and compare sequence models for character-level name generation using recurrent neural architectures, evaluating novelty, diversity, and realism of generated names.

2.2 Task 0: Dataset

2.2.1 Dataset Description

1000 Indian names were generated using Large Language Models (LLMs) and stored in `data/TrainingNames.txt`.

Table 14: Name Dataset Statistics

Metric	Value
Total Names	1,000
Training Examples	1,000
Character Vocabulary Size	24
Unique Characters	21 (a-y, excluding f, q, w, x, z)
Special Tokens	3 (<pad>, <bos>, <eos>)

Character Set: `a, b, c, d, e, g, h, i, j, k, l, m, n, o, p, r, s, t, u, v, y`

Sample Names: Aadesh, Aadika, Aadish, Aadita, Aadra, Aarav, Aaren, Aarika, Aarish, Aarita, Adiansh, Amajeet, Amiendra, Anideep, Mukendra, Ashendra, Varini, Divveer

2.2.2 Character-Level Encoding

The dataset was prepared for autoregressive next-character prediction:

- **Input Sequence:** <bos> + name characters
- **Target Sequence:** name characters + <eos>
- **Padding:** Variable-length sequences padded to batch maximum length

2.3 Task 1: Model Implementation

2.3.1 Architecture 1: Vanilla RNN

Description: A simple recurrent neural network implemented from scratch using custom RNN cells with tanh activation.

Architecture Details:

Listing 3: Vanilla RNN Architecture

```
Input: Character indices (batch_size, seq_len)
       |
Embedding Layer: vocab_size x hidden_size
       |
RNN Cells (num_layers): h_t = tanh(W_ih * x_t + W_hh * h_{t-1})
       |
Dropout: p=0.3
       |
Linear Decoder: hidden_size -> vocab_size
       |
Output: Character probabilities (batch_size, seq_len, vocab_size)
```

Custom RNN Cell Implementation:

```python
class ManualRNNCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        self.in_linear = nn.Linear(input_size, hidden_size, bias=True)
        self.h_linear = nn.Linear(hidden_size, hidden_size, bias=False)

    def forward(self, x_t, h_prev):
        return torch.tanh(self.in_linear(x_t) + self.h_linear(h_prev))
```

Trainable Parameters: ∼19,000

2.3.2 Architecture 2: Bidirectional LSTM (BiLSTM)

Description: Bidirectional LSTM processes sequences in both forward and backward directions, capturing context from both sides.

Architecture Details:

Listing 4: BiLSTM Architecture

```
Input: Character indices (batch_size, seq_len)
       |
Embedding Layer: vocab_size x hidden_size
       |
Forward LSTM Cells: Process left-to-right
Backward LSTM Cells: Process right-to-left
       |
Concatenate: [forward_output; backward_output] -> 2*hidden_size
       |
Dropout: p=0.3
       |
Linear Decoder: 2*hidden_size -> vocab_size
       |
Output: Character probabilities
```

LSTM Gate Equations:

$$i_t = \sigma(W_{ii}x_t + W_{hi}h_{t-1} + b_i) \quad \text{(Input gate)} \tag{2}$$
$$f_t = \sigma(W_{if}x_t + W_{hf}h_{t-1} + b_f) \quad \text{(Forget gate)} \tag{3}$$
$$g_t = \tanh(W_{ig}x_t + W_{hg}h_{t-1} + b_g) \quad \text{(Cell candidate)} \tag{4}$$
$$o_t = \sigma(W_{io}x_t + W_{ho}h_{t-1} + b_o) \quad \text{(Output gate)} \tag{5}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \tag{6}$$
$$h_t = o_t \odot \tanh(c_t) \tag{7}$$

Trainable Parameters: ∼161,000

2.3.3 Architecture 3: RNN with Attention

Description: An LSTM-based RNN augmented with scaled dot-product attention mechanism that allows the model to focus on relevant previous characters.

Architecture Details:

Listing 5: RNN with Attention Architecture

```
Input: Character indices (batch_size, seq_len)
       |
Embedding Layer: vocab_size x hidden_size
       |
LSTM Cells: Process sequence, store all hidden states
       |
Attention Mechanism:
  - Query: Current hidden state projected
  - Keys: All previous hidden states projected
  - Values: All previous hidden states projected
  - Attention Score: softmax(Q * K^T / sqrt(d_k))
  - Context: Attention x Values
       |
Combine: [hidden_state; context_vector]
       |
Linear Decoder: (hidden_size + attention_dim) -> vocab_size
       |
Output: Character probabilities
```

Attention Computation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{8}$$

Trainable Parameters: ∼131,000

2.3.4 Hyperparameters Summary

Table 15: RNN Training Hyperparameters

Parameter	Value
Hidden Size	128
Number of Layers	1
Learning Rate	0.001
Dropout	0.3
Batch Size	32
Epochs	30
Optimizer	Adam
Gradient Clipping	1.0
Attention Dimension	64

2.4 Task 2: Quantitative Evaluation

2.4.1 Generation Methodology

Names were generated autoregressively:

1. Start with `<bos>` token
2. Predict next character using temperature-scaled softmax sampling ($T = 1.05$)
3. Apply top-k sampling ($k = 10$) with repetition penalty
4. Stop at `<eos>` or maximum length (14 characters)
5. Filter invalid outputs (too short, too long, no vowels, excessive repetition)

2.4.2 Evaluation Metrics

Table 16: Name Generation Quantitative Results

Model	Generated	Unique	Novelty Rate	Diversity
BiLSTM	118	118	**1.000**	**1.000**
Vanilla RNN	111	111	0.604	1.000
RNN with Attention	117	117	0.530	1.000

Metric Definitions:

- **Novelty Rate:** Percentage of generated names NOT appearing in training set
- **Diversity:** Number of unique names / Total generated names

2.4.3 Analysis

1. **BiLSTM achieves perfect novelty (100%):**
   - The bidirectional context helps generate more creative combinations
   - Less prone to memorizing training sequences exactly
2. **All models achieve perfect diversity (100%):**
   - Temperature and top-k sampling successfully introduces variation
   - Repetition penalty prevents duplicate outputs
3. **Vanilla RNN and Attention models show lower novelty:**
   - More likely to reproduce training patterns
   - Attention mechanism may over-focus on exact training sequences

2.5 Task 3: Qualitative Analysis

2.5.1 Representative Generated Samples

Table 17: Vanilla RNN Generated Names (Sample)

#	Name	#	Name
1	Sheayr	11	Praika
2	Ramva	12	Mukish
3	Tanika	13	Shina
4	Sanini	14	Samir
5	Reeav	15	Subdeep
6	Nirsa	16	Tarans
7	Sarraj	17	Sruen
8	Amajeet	18	Dhian
9	Kalindra	19	Sidra
10	Darya	20	Prail

Table 18: BiLSTM Generated Names (Sample)

#	Name	#	Name
1	Yoog	11	Jeta
2	Shiseep	12	Jyoggpr
3	Gilul	13	Guril
4	Paran	14	Yoknd
5	Gamut	15	Ubbey
6	Prub	16	Chait
7	Nived	17	Subir
8	Nivdiv	18	Yukkansr
9	Chir	19	Numish
10	Gokubs	20	Dokiv

Table 19: RNN with Attention Generated Names (Sample)

#	Name	#	Name
1	Sheans	11	Mihesh
2	Arula	12	Ramkish
3	Aviesh	13	Meeraj
4	Nisvaya	14	Arit
5	Shrul	15	Dakum
6	Arum	16	Mukendra
7	Kriali	17	Ashendra
8	Vaivra	18	Lakni
9	Vinali	19	Ayuen
10	Malika	20	Sheni

2.5.2 Realism Assessment

Vanilla RNN:

- Produces mostly realistic Indian names (Tanika, Sanini, Samir, Sidra, Kalindra)
- Some phonetically plausible but uncommon combinations (Sheayr, Praika, Prail)
- Good understanding of common Indian name patterns (-deep, -jeet endings)

BiLSTM:

- More experimental and creative names
- Some unrealistic outputs (Jyoggpr, Yukkansr, Gokubs)
- Higher novelty comes at cost of realism
- Produces some valid names (Subir, Chait, Nived)

RNN with Attention:

- Best balance between novelty and realism
- Many authentic-sounding names (Malika, Vinali, Mihesh, Ashendra, Mukendra)
- Captures complex Indian name patterns (-endra, -esh, -ali suffixes)
- Attention mechanism helps preserve name structure

2.5.3 Failure Mode Analysis

Table 20: Failure Mode Statistics

Model	**Generated**	**RepeatRun≥3 (%)**	**DoubleChar (%)**	**NoVowel (%)**	**AvgLe**
Vanilla RNN	111	0.00	19.82	0.00	5.66
BiLSTM	118	0.00	24.58	0.00	5.42
RNN+Attention	117	0.00	15.38	0.00	5.78

Observations:

1. **No Repetition Runs (≥3):** All models successfully avoid excessive character repetition
2. **Double Characters:** BiLSTM has highest double-char rate (24.58%); RNN with Attention has lowest (15.38%)
3. **Vowel Presence:** All models maintain vowel presence in 100% of outputs
4. **Average Length:** All models produce names averaging 5.4-5.8 characters

2.5.4 Common Failure Modes by Model

Vanilla RNN:

- Occasional truncated endings (Tenav, Prail)
- Sometimes produces names too similar to training data

BiLSTM:

- Consonant clusters (Jyoggpr, Yukkansr, Gokubs)
- Overly creative combinations that lack phonetic naturalness

RNN with Attention:

- Can over-memorize patterns from training data (lower novelty)
- Occasionally produces incomplete-sounding names (Shrul, Arum)

3 Conclusion

3.1 Problem 1: Word Embeddings

- Successfully collected and preprocessed 70,637 tokens from IIT Jodhpur sources
- Both CBOW and Skip-gram models were trained with various hyperparameters
- Best CBOW configuration: embed_dim=200, window=5, lr=0.01 (loss: 0.2596)
- Best Skip-gram configuration: embed_dim=100, window=2, neg=15, lr=0.001 (loss: 0.2840)
- Word analogy results were mixed, highlighting challenges with domain-specific corpora
- Skip-gram showed better performance on the engineering-science analogy

3.2 Problem 2: Name Generation

- Implemented three RNN variants from scratch (Vanilla RNN, BiLSTM, RNN with Attention)
- BiLSTM achieves highest novelty (100%) but lowest realism
- RNN with Attention provides best balance of novelty and name quality
- All models achieve 100% diversity with proper sampling techniques
- Character-level modeling successfully captures Indian name phonotactics

3.3 Key Learnings

1. **Word Embeddings:**
   - Domain-specific corpora (academic content) produce domain-relevant embeddings
   - Hyperparameter tuning significantly impacts embedding quality
   - Skip-gram better captures rare word relationships
2. **Sequence Generation:**
   - Bidirectional context increases creativity but may reduce realism
   - Attention mechanisms help maintain structural coherence
   - Proper sampling techniques (temperature, top-k) crucial for quality output
   - Validation filters essential for removing degenerate outputs